

TypeScript and React Models

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 3

Who this book is for . . . 3

A book that stands on its own . . . 3

What you need . . . 4

Conventions . . . 4

I: The Why . . . 6

1. JavaScript as another bytecode . . . 7

1.1. Not a new idea . . . 7

1.2. A question of direction . . . 7

1.3. Data as a versioned module . . . 8

2. Why a model, not a template . . . 9

2.1. Import management . . . 9

2.2. Many consumers, not one . . . 10

2.3. A transformation between models . . . 10

2.4. Generated code is marked . . . 10

3. Where the models stop . . . 11

3.1. The declaration surface, not the type algebra . . . 11

3.2. What React leaves opaque . . . 11

3.3. Write only . . . 11

3.4. Ownership through markers . . . 11

II: The How . . . 13

4. Getting started . . . 14

4.1. Four modules . . . 14

4.2. Adding the models . . . 14

4.3. What the build does . . . 15

5. The TypeScript model . . . 16

5.1. Imports, computed . . . 17

5.2. The generator . . . 19

6. The React model . . . 20

6.1. A component in two syntaxes . . . 20

6.2. HTML, shared . . . 24

6.3. The generator . . . 25

7. Component libraries . . . 26

7.1. React Bootstrap . . . 26
7.2. React Flow . . . 27
8. Applications . . . **30**
8.1. Static or served . . . 30
8.2. Walking through an application . . . 31
III: Looking Ahead . . . **33**
9. Looking ahead . . . **34**
9.1. A UI model, drawn . . . 34
9.2. The polymorphic UI . . . 34
9.3. Vocabularies as npm packages . . . 34
9.4. One source, many audiences . . . 34
9.5. Vendored modules . . . 35
9.6. Documentation for npm users . . . 35
Appendix A: Reference . . . **36**
A.1. Packages . . . 36
A.2. Enumerations of the TypeScript model . . . 36
A.3. Java classes . . . 36
Appendix B: Building this book . . . **38**
B.1. Generated code . . . 38
B.2. Compiling the generated TypeScript . . . 38
B.3. The figure . . . 39
Appendix C: Glossary . . . **40**

Colophon

TypeScript and React Models: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes four models of the Nasdanika SDK Runtime 2026.11.0: the TypeScript model, Maven coordinates `org.nasdanika.sdk.runtime.models.typescript:model:2026.11.0`, namespace URI `runtime.sdk.nasdanika.org/models/typescript`; the React model, `org.nasdanika.sdk.runtime.models.react:model:2026.11.0`, namespace URI `runtime.sdk.nasdanika.org/models/react`; and the React Bootstrap and React Flow models, `org.nasdanika.sdk.runtime.models.react:bootstrap` and `org.nasdanika.sdk.runtime.models.react:flow`. Every listing in it is included from the tests of those modules, which are compiled and run when the book is built, and every block of generated code is a file that a test compares with what the generator writes. Its figure is a screenshot that a test took of a generated application in a browser.

Copyright © 2026 Pavel Vlasov.

The text and figures of this book are licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt them for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *TypeScript and React Models: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

TypeScript is a trademark of Microsoft Corporation. React is a trademark of Meta Platforms. React Flow is by xyflow. Bootstrap is released by its authors under the MIT License. Node.js is a trademark of the OpenJS Foundation. Eclipse and EMF are trademarks of the Eclipse Foundation. Java is a trademark of Oracle and its affiliates. Other names may be trademarks of their owners. Their use here identifies the technologies the models relate to and implies no endorsement.

The source of this book and the React model's tests are at github.com/Nasdanika-SDK/runtime, in `models/react/model`; the TypeScript model is in `models/typescript/model`, and the React Bootstrap and React Flow models are in `models/react/bootstrap` and `models/react/flow`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books.

Preface

This book is about browser applications generated from Java. It covers four models of the Nasdanika SDK Runtime: TypeScript's declaration surface, React's components, hooks and JSX on top of it, and two libraries of React components, React Bootstrap and React Flow, as annotated models. Their generators write code that a browser runs as it is: no npm, no Node.js and no build step, and still applications, because React provides the dynamic behavior.

The first part of the book is the why: JavaScript as another bytecode, why code is generated through models and not templates, and where the models stop on purpose. The second part is the how, for Java developers: import management in TypeScript, a React component in two syntaxes, component libraries, and applications that are served, written as static sites, and walked through in a browser. The third says what comes next.

Who this book is for

Java developers who need a browser application, a viewer of their models or data, a table, a diagram, a dashboard, and want to build it with Java's tools.

Writers of generators who emit TypeScript or React from models, and want the imports, the provenance and the structure that a template loses.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. It covers four modules, because they are used together: React builds on TypeScript, and the libraries on React. *HTML and Bootstrap Models* is close to it: HTML elements and the Bootstrap model appear in JSX here. You do not need that book, or any other book of the series, to read this one: where the books touch, this one repeats what you need.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The modules are `org.nasdanika.sdk.runtime.models.typescript`, `org.nasdanika.sdk.runtime.models.react`, `org.nasdanika.sdk.runtime.models.react.bootstrap` and `org.nasdanika.sdk.runtime.models.react.flow`.
- Some familiarity with TypeScript and React. Some familiarity with EMF helps: an EMF model is a set of Java interfaces generated from a metamodel, with containment, references, reflection and persistence for free.
- For the walk-through of Chapter 8, Chrome, or Playwright's Chromium, and access to the internet.

Conventions

- **Every listing is included from a test** and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **Generated code is checked.** Each block of generated TypeScript, JSX or JavaScript is a file in the tests, compared with what the generator writes.
- **Status labels** mark what this edition does not have yet: PLANNED for designed but not built, SKETCH for partly built.
- **Voice.** I write in the first person.

> Editor's notes record inconsistencies between the models and their documentation found while writing. They are shown in builds for the author and left out of published editions.
>
> This book replaces the readmes of the TypeScript and React models, `models/typescript/model/doc/readme.md`, `models/typescript/README.md` and `models/react/model/doc/readme.md`, which are deleted. The readmes referred to figures, `typescript.drawio` and `react.drawio`, which are in the modules and are not in this book. For the book, the expected outputs of `ReactGeneratorTests` moved from text blocks to files in `src/test/resources/expected`, and long lines of the tagged tests were wrapped; the output is the same. The Javadoc of `JvmPropertiesTableTests` says the React model has no emitter yet; `ReactGenerator` is

> that emitter.

Part I: The Why

Why browser applications should be generated from Java through models, why a model and not a template, and where the models stop. No code is needed to read this part.

Chapter 1. JavaScript as another bytecode

A React application is usually a Node.js project: a package manifest, hundreds of packages in `node_modules`, a bundler and a build. None of it is needed to run React in a browser. Browsers load ES modules natively, an import map in the page resolves bare specifiers such as `react` to URLs, and a CDN such as esm.sh serves npm packages as ES modules. What remains is the application's own code, and the runtime generates that from models.

So a Java developer builds a browser application with Java's APIs and models, with no npm, no Node.js and no build step, and with little knowledge of how React works. What runs in the browser can be deployed as static files with no JVM behind it, and still be an application.

1.1. Not a new idea

With the TypeScript and React models, JavaScript is a target of generation: the format a machine executes, written by a tool from a source people work in, as bytecode is written by a compiler. Google Web Toolkit compiled Java to JavaScript in 2006, and J2CL, TeaVM, Kotlin/JS and Scala.js do it today. The runtime's route is different, generation from models into readable modules, not compilation of Java code, but the destination is the same: people work on the JVM, and the browser gets JavaScript.

1.2. A question of direction

On the JVM, with these generators, a team can do most of what it would do with Node.js to build and serve a browser application: generate modules and pages, serve them, test them in a browser, and deploy them as static files. Not all of it: bundlers, React rendered on the server, and the tool chain of linters and component test runners run on Node.js, and only some of them run on the JVM. The other direction works as text, since a Node.js program can write Java source as easily as Java writes JavaScript. What it cannot do is be the JVM: load EMF, JDBC drivers and the rest of the enterprise ecosystem into its own process. The JVM can generate and host most of a browser application; Node.js can generate Java but cannot host what Java runs on.

1.3. Data as a versioned module

Applications are one output; data is the other. Models published as Maven artifacts need a JVM to be used. The TypeScript model is how the same models, and the data loaded into them, reach consumers that have no JVM and never will: browsers, Node.js tooling and React applications.

The payload is not a REST endpoint or a JSON document fetched at run time. It is an npm package, and that choice carries most of the value **PLANNED**:

- **Types and data ship together.** One generation emits the interfaces and the data that satisfies them, so `tsc` checks every access against the schema the data actually has.
- **Versions are snapshots.** `2026.9.0` is the data as of that release, immutable and installable next to the one before it, so "how far has this drifted" is a comparison of two packages.
- **Distribution is a solved problem.** Registries, scopes, integrity hashes, lockfiles and mirrors exist and are operated by people whose job that is.
- **No back end.** An application that imports its reference data has no API to stand up and no reason for anything to leave the machine.
- **Provenance survives the trip.** Every generated element can carry markers back to what it was generated from.

In the browser, the stance is viewer and analyzer first. Authoring stays with the sources.

Chapter 2. Why a model, not a template

Emitting TypeScript declarations from a model with a template is two hundred lines and a day's work. If that were the only consumer, a template would win. Four things carry the models instead.

2.1. Import management

This is the reason the TypeScript model exists. A generator has to get all of this right, for every reference, every time:

- **Named, default and namespace imports**: three statements with three rules for local names.
- `import type` **or a value import**, which decides whether the import is erased when compiled. Get it wrong across generated packages, and a cycle between packages stops compiling, or a package that needed only another's types drags in its run time.
- **Relative or bare specifiers**: a file in this package, or a dependency that must appear in `package.json`.
- **The rule of** `NodeNext`, under which a relative specifier of an ES module has a `.js` extension even in `.ts` source.
- **Names that collide**: two types named `Account` from two modules, one of which must be aliased.

Generators written by hand get this wrong repeatedly. In the model, a module's imports are computed when it is generated, from the type references reachable in it: a reference points at its declaration, and the generator walks from the declaration to its module and package, and decides the specifier and whether the import can be type-only. Imports written by hand remain for what the generator cannot see: imports for side effects, and types used only inside opaque text.

React adds components used inside JSX to the problem: a `<Badge>` in a render function is an import too. The React model's JSX elements reference their components, so their imports are computed the same way.

2.2. Many consumers, not one

Models to TypeScript is the first consumer, not the only one. The React model extends the TypeScript model, and tool definitions and their schemas for agents are TypeScript too. So the TypeScript model is built as a good general target, not one shaped like EMF.

2.3. A transformation between models

A source model becomes a TypeScript or React model and then text, instead of becoming text directly. The result in the middle is a graph of objects that can be validated, queried, compared and reported on before a character is written.

2.4. Generated code is marked

Every element of the TypeScript model is a `NamedElement` of the runtime's core model, so every module, declaration and member can carry markers back to the source elements it came from, and documentation, which the generator writes as JSDoc. A constant in a browser bundle can still answer which spreadsheet cell or which database column produced it.

Chapter 3. Where the models stop

3.1. The declaration surface, not the type algebra

Modeling all of TypeScript would take several hundred classes and would still trail a language that adds features to its type system every quarter. The TypeScript model covers what a generator needs to write a declaration, about thirty classes, and stops, with three escape hatches:

- `OpaqueTypeReference`: conditional types, mapped types, template literal types, `keyof` and `typeof` constructions.
- `OpaqueMember`: index signatures, call signatures, sets of overloads, awkward decorators.
- `CodeBlock`: the bodies of functions and methods, initializers, default values.

All three have a `references` list, and that detail makes them safe: opaque text still declares the named types used in it, so an opaque block does not defeat import management.

3.2. What React leaves opaque

Hook semantics, suspense, transitions, server components, error boundaries and routing are not modeled. They arrive as `OpaqueHook` and `OpaqueJsx`, with `references`, which keeps the React model at about thirty classes instead of chasing a framework that moves every quarter.

3.3. Write only

There is no loader, and probably never will be. Parsing TypeScript means the TypeScript compiler API, which is TypeScript, not Java, and a second-rate parser in Java for a language this size is not worth starting. If reverse engineering is ever wanted, it belongs on the TypeScript side, emitting the model as JSON. The omission is a decision, not a gap.

3.4. Ownership through markers

The models sit on the runtime's core model and need nothing from models of architecture, ownership or governance. Ownership arrives through markers, not supertypes: a generated module points at the source elements it was generated from, and those are owned and

governed. Making every JSX attribute an owned object would attach ownership to the artifact instead of to its cause.

Part II: The How

The models in Java: adding them to a project, TypeScript and its import management, React components with hooks and JSX in two syntaxes, component libraries as annotated models, and applications that are served, static, and walked through in a browser.

Chapter 4. Getting started

4.1. Four modules

Module	What it has
`org.nasdanika.sdk.runtime.models.typescript`	The TypeScript model and `Generator`, which writes modules and packages with managed imports to a volume
`org.nasdanika.sdk.runtime.models.react`	The React model and `ReactGenerator`, a TypeScript generator that also writes components, hooks, contexts and JSX
`org.nasdanika.sdk.runtime.models.react.bootstrap`	React Bootstrap's components, for layout, content, navigation, overlays and forms
`org.nasdanika.sdk.runtime.models.react.flow`	React Flow, `@xyflow/react` 12: flows of nodes and edges, the canvas and its plugins, hooks, and `LayeredLayout`, a simple layout to start from

4.2. Adding the models

The runtime's models have a bill of materials, `org.nasdanika.sdk.runtime.models:bom`, which manages their versions. Import it in `dependencyManagement`, and depend on what you use, without versions:

```
<dependency>
    <groupId>org.nasdanika.sdk.runtime.models.react</groupId>
    <artifactId>model</artifactId>
</dependency>
<dependency>
    <groupId>org.nasdanika.sdk.runtime.models.react</groupId>
    <artifactId>bootstrap</artifactId>
</dependency>
```

The React model brings the TypeScript model and the runtime's HTML model; React Bootstrap brings the runtime's HTML Bootstrap model too.

Listing 1. The module declaration of the TypeScript model

```
module org.nasdanika.sdk.runtime.models.typescript {

    exports org.nasdanika.sdk.runtime.models.typescript;
```

```
    exports org.nasdanika.sdk.runtime.models.typescript.impl;
    exports org.nasdanika.sdk.runtime.models.typescript.util;
    exports org.nasdanika.sdk.runtime.models.typescript.generator;

    requires transitive org.nasdanika.sdk.runtime.models.core;
    requires transitive org.nasdanika.sdk.runtime.volume;

    provides EPackage with TypescriptEPackageProvider;

}
```

The TypeScript model requires the runtime's core model, for documentation and markers, and its volume module: the generator writes to any volume of text.

Listing 2. The module declaration of the React model

```
module org.nasdanika.sdk.runtime.models.react {

    exports org.nasdanika.sdk.runtime.models.react;
    exports org.nasdanika.sdk.runtime.models.react.impl;
    exports org.nasdanika.sdk.runtime.models.react.util;
    exports org.nasdanika.sdk.runtime.models.react.generator;

    requires transitive org.nasdanika.sdk.runtime.models.typescript;
    requires transitive org.nasdanika.sdk.runtime.models.html;
    requires java.net.http;
    requires jdk.httpserver;

    provides EPackage with ReactEPackageProvider;

}
```

4.3. What the build does

The Xcore definitions are `typescript.xcore`, `react.xcore`, the five packages of React Bootstrap, and `flow.xcore`. Maven generates the Java sources into `src-gen` with the Xtext Maven plugin. The generators and `LayeredLayout` are hand-written. Nothing in the build runs Node.js; compiling generated TypeScript with `tsc` is a separate, optional step (Appendix B).

Chapter 5. The TypeScript model

Area	Classes
Base	`TypeScriptElement`, a core model `NamedElement`
Packaging	`Package`, `Module`
Imports	`Import`, `ImportBinding`, `ImportKind`
Type references	`TypeReference`, `PrimitiveTypeReference`, `NamedTypeReference`, `ArrayTypeReference`, `UnionTypeReference`, `IntersectionTypeReference`, `LiteralTypeReference`, `TypeLiteralReference`, `FunctionTypeReference`, `OpaqueTypeReference`
Declarations	`Declaration`, `InterfaceDeclaration`, `ClassDeclaration`, `TypeAliasDeclaration`, `EnumDeclaration`, `EnumMember`, `FunctionDeclaration`, `VariableDeclaration`
Signatures	`Callable`, `Parameter`, `TypeParameter`, `CodeBlock`
Members	`Member`, `Property`, `Method`, `Constructor`, `Accessor`, `OpaqueMember`

A `Package` is deliberately thin: a scoped `packageName`, a `version`, a description, a license, the module type, dependencies, and its `modules`, enough to write `package.json` and lay out files. It is there because a generator that writes modules but not the package around them leaves the last mile to a script, and that script is where drift starts.

A `Module` has a `path` in the package, `declarationOnly` for a `.d.ts`, a header, its authored `imports`, and its `declarations`. A `Declaration` may be `exported` or `ambient`.

`Callable` is the supertype of functions, methods, constructors and accessors, so the generator writes one signature routine for all four: type parameters, parameters, a return type, a body, `async` and `generator`.

A `NamedTypeReference` points at a declaration of the model through `target`, or names a type the model does not own by `qualifiedName` and `moduleSpecifier`, with `typeArguments`. The target is what the import resolver follows.

5.1. Imports, computed

The test of this chapter builds a small package of a bank: a module of common types, a module of the bank's model, and a module of a legacy system with a type of the same name.

Listing 3. A package of three modules (`ImportManagementTests.java`, in the TypeScript model)

```
Package bank = TS.createPackage();
bank.setName("bank");
bank.setPackageName("@example/bank");
bank.setVersion("1.0.0");
bank.getDependencies().add("decimal.js@^10.4.3");                  ①

Module common = module(bank, "model/common");
EnumDeclaration currency = TS.createEnumDeclaration();
currency.setName("Currency");
currency.setExported(true);
currency.getMembers().add(member("USD"));
currency.getMembers().add(member("EUR"));
common.getDeclarations().add(currency);
InterfaceDeclaration money = type(common, "Money");
property(money, "amount", external("Decimal", "decimal.js"));      ②
property(money, "currency", named(currency));

Module model = module(bank, "model/bank");
InterfaceDeclaration account = type(model, "Account");
property(account, "id", primitive(PrimitiveType.STRING));
property(account, "balance", named(money));
ClassDeclaration base = TS.createClassDeclaration();
base.setName("AbstractAccount");
base.setExported(true);
base.setAbstract(true);
base.getInterfaces().add(named(account));
model.getDeclarations().add(base);

Module legacy = module(bank, "model/legacy");
InterfaceDeclaration legacyAccount = type(legacy, "Account");         ③
property(legacyAccount, "number", primitive(PrimitiveType.STRING));
```

① A dependency of the package, written into `package.json`.

② A type of a package the model does not own, named by its module specifier.

③ Another `Account`, in another module.

A fourth module uses them, and its imports are not written by hand:

Listing 4. A module whose imports are computed (`ImportManagementTests.java`)

```
Module service = module(bank, "service/accounts");
```

```
FunctionDeclaration open = TS.createFunctionDeclaration();
open.setName("open");
open.setExported(true);
Parameter id = TS.createParameter();
id.setName("id");
id.setType(primitive(PrimitiveType.STRING));
open.getParameters().add(id);
open.setReturnType(named(account));                              ①
CodeBlock body = TS.createCodeBlock();
body.setText("""
        const opened = new CheckingAccount();
        console.log(`Opened ${id} in ${Currency.USD}`);
        return opened;""");
body.getReferences().add(named(currency));                       ②
open.setBody(body);
service.getDeclarations().add(open);

ClassDeclaration checking = TS.createClassDeclaration();
checking.setName("CheckingAccount");
checking.setExported(true);
checking.setSuperClass(named(base));                             ③
Property migrated = TS.createProperty();
migrated.setName("migratedFrom");
migrated.setOptional(true);
migrated.setType(named(legacyAccount));                          ④
checking.getMembers().add(migrated);
service.getDeclarations().add(checking);

String generated = new Generator().generate(service);
```

① `Account` is used only as a type.

② The body is opaque text, and it uses `Currency` as a value. Its references say so.

③ A superclass is evaluated at run time: a value import.

④ The legacy `Account`, used only as a type.

The generator writes, and the test compares with this file:

Listing 5. The generated module (`service/accounts.ts`)

```
import { Currency } from "../model/common.js";
import { AbstractAccount } from "../model/bank.js";
import type { Account } from "../model/bank.js";
import type { Account as Account$2 } from "../model/legacy.js";

export function open(id: string): Account {
  const opened = new CheckingAccount();
  console.log(`Opened ${id} in ${Currency.USD}`);
  return opened;
}
```

```
export class CheckingAccount extends AbstractAccount {
  migratedFrom?: Account$2;
}
```

`Currency` and `AbstractAccount` are value imports, because a code block and a superclass need them at run time. The bank's `Account` is a type import, erased when compiled. The legacy `Account` came second and is aliased. Specifiers are relative, with the `.js` extension `NodeNext` requires.

5.2. The generator

`Generator` writes a `Package` or a `Module` to a `GenericVolume<String>`, the runtime's abstraction of a tree of files, here of text: a directory adapted to text, an in-memory volume for a test, or any other volume. For a package it writes `package.json`, `tsconfig.json`, and a file per module under `src`; missing directories are created through the volume. `generate(module)` returns one module as a string.

The module's `BankTests` generate a larger package that uses every construct of the model into `target/bank`. It compiles with `tsc` under `strict` and `--verbatimModuleSyntax`, which turns a wrong decision between a type and a value import into a compile error, in a step outside the Maven build (Appendix B).

Chapter 6. The React model

The TypeScript model leaves bodies as opaque code blocks, with no JSX, on purpose. Generating React through it alone would put every component's body into opaque text and lose import management, provenance and structure. `CodeBlock` has a `references` list so that a structured body could be substituted; the React model's `JsxBlock` is that substitution.

Area	Classes
Base	`ReactElement`, `TsxModule`
Components	`ComponentDeclaration`, a TypeScript function declaration; `JsxBlock`
Hooks	`Hook`, `StateHook`, `EffectHook`, `MemoHook`, `ContextHook`, `RefHook`, `OpaqueHook`
Context	`ContextDeclaration`, `ContextProvider`
JSX	`JsxNode`, `JsxContainer`, `JsxElement`, `JsxFragment`, `JsxText`, `JsxExpression`, `JsxConditional`, `JsxIterator`, `OpaqueJsx`
HTML	`Intrinsic`, an element of the HTML model used as a JSX intrinsic; `JsxContent`, JSX inside an HTML container
Component libraries	`LibraryComponent`, `LibraryHook`, `ValueBlock`
Attributes	`JsxAttribute`, `LiteralAttribute`, `ExpressionAttribute`, `EventAttribute`, `SpreadAttribute`

`JsxElement.componentType` is the feature everything turns on: a reference to the component's declaration, which the generator follows to decide which components to import, from which module, and whether the import can be type-only.

6.1. A component in two syntaxes

The listings of this chapter build a package of two modules: a `Badge` component, and a `UserList` that filters users by name and shows each with a badge. They use a small helper of the tests, `Jsx`: each of its methods creates one object of the React, TypeScript or HTML model and sets what the call passes. The same builder produces either syntax: JSX in a `TsxModule`, with type annotations, or `createElement` calls in a plain module, without them.

Listing 6. A component (ReactGeneratorTests.java)

```
ComponentDeclaration badge = REACT.createComponentDeclaration();
badge.setName("Badge");
badge.setExported(true);
badge.getParameters().add(Jsx.parameter("props"));
badge.setRender(Jsx.children(
        Jsx.tag("span", Jsx.literal("className", "badge")),
        Jsx.expression("props.name")));
badgeModule.getDeclarations().add(badge);
```

A context, with a type in TypeScript:

Listing 7. A context (ReactGeneratorTests.java)

```
ContextDeclaration themeContext = REACT.createContextDeclaration();
themeContext.setName("ThemeContext");
themeContext.setExported(true);
if (tsx) {
    themeContext.setValueType(primitive(PrimitiveType.STRING));
}
themeContext.setDefaultValue(Jsx.code("\"light\""));
module.getDeclarations().add(themeContext);
```

Hooks are model objects: state, a context, and a memoized value with its dependencies.

Listing 8. Hooks (ReactGeneratorTests.java)

```
StateHook filter = Jsx.state("filter", "setFilter", "\"\"");
if (tsx) {
    filter.setStateType(primitive(PrimitiveType.STRING));
}
userList.getHooks().add(filter);

ContextHook theme = REACT.createContextHook();
theme.getBindings().add("theme");
theme.setContext(themeContext);
userList.getHooks().add(theme);

MemoHook visible = REACT.createMemoHook();
visible.getBindings().add("visible");
visible.setExpression(Jsx.code("props.users.filter(user => user.name.includes(filter))"));
visible.getDependencies().add(Jsx.code("props.users"));
visible.getDependencies().add(Jsx.code("filter"));
userList.getHooks().add(visible);
```

The render is a tree of JSX. HTML elements are elements of the runtime's HTML model, used as intrinsics, with JSX attributes for what is computed; `JsxContent` puts JSX back inside an HTML

element, as the badge inside the list item:

Listing 9. The render (`ReactGeneratorTests.java`)

```
Div root = HTML.createDiv();
root.getClasses().add("user-list");
root.getStyle().put("margin-top", "1rem");
root.getAttributes().put("data-theme", "light");

Label label = HTML.createLabel();
label.setHtmlFor("filter");
Jsx.contents(label, Jsx.htmlText("Filter: "));

Input input = HTML.createInput();
input.setId("filter");
input.setType("text");
input.setValue("ignored, value is controlled");

ListItem item = HTML.createListItem();
Jsx.contents(
        item,
        Jsx.content(Jsx.element(Jsx.reference(badge), Jsx.attribute("name", "user.name"))),
        Jsx.htmlText(" & friends"));

Paragraph empty = HTML.createParagraph();
Jsx.contents(empty, Jsx.htmlText("No users"));

ContextProvider provider = REACT.createContextProvider();
provider.setContext(themeContext);
provider.setValue(Jsx.code("theme"));
userList.setRender(Jsx.children(
        provider,
        Jsx.children(
                Jsx.intrinsic(root, Jsx.attribute("data-theme", "theme")),
                Jsx.intrinsic(label),
                Jsx.intrinsic(
                        input,
                        Jsx.attribute("value", "filter"),
                        Jsx.event("onChange", "event => setFilter(event.target.value)")),
                Jsx.children(
                        Jsx.intrinsic(HTML.createUnorderedList()),
                        Jsx.iterator("visible", "user", "user.id", Jsx.intrinsic(item))),
                Jsx.conditional("visible.length === 0", Jsx.intrinsic(empty), null))));
module.getDeclarations().add(userList);
```

In JSX syntax, the generator writes:

Listing 10. `users.tsx`

```
import { Badge } from "./components/badge.js";
import { createContext, memo, useState, useContext, useMemo } from "react";
```

```
export const ThemeContext = createContext<string>("light");

export interface UserListProps {
  users: { id: number; name: string }[];
}

/**
 * Users, filtered by name.
 */
export const UserList = memo(function UserList(props: UserListProps) {
  const [filter, setFilter] = useState<string>("");
  const theme = useContext(ThemeContext);
  const visible = useMemo(() => (props.users.filter(user => user.name.includes(filter))), [props
.users, filter]);
  return (
    <ThemeContext.Provider value={theme}>
      <div className="user-list" data-theme={theme} style={{ marginTop: "1rem" }}>
        <label htmlFor="filter">Filter: </label>
        <input
          id="filter"
          type="text"
          value={filter}
          onChange={event => setFilter(event.target.value)}
        />
        <ul>
          {visible.map(user => <li key={user.id}><Badge name={user.name} />{" & friends"}</li>)}
        </ul>
        {(visible.length === 0) && <p>No users</p>}
      </div>
    </ThemeContext.Provider>
  );
});

function renderName(name) {
  const upper = name.toUpperCase();
  return <b>{upper}</b>;
}
```

And in a plain module, `createElement` calls that a browser runs as they are:

Listing 11. `users.js`

```
import { Badge } from "./components/badge.js";
import { createContext, memo, useState, useContext, useMemo, createElement } from "react";

export const ThemeContext = createContext("light");

/**
 * Users, filtered by name.
 */
export const UserList = memo(function UserList(props) {
```

```
  const [filter, setFilter] = useState("");
  const theme = useContext(ThemeContext);
  const visible = useMemo(() => (props.users.filter(user => user.name.includes(filter))), [props
.users, filter]);
  return createElement(ThemeContext.Provider, { value: theme },
    createElement("div", {
      className: "user-list",
      "data-theme": theme,
      style: { marginTop: "1rem" }
    },
      createElement("label", { htmlFor: "filter" }, "Filter: "),
      createElement("input", {
        id: "filter",
        type: "text",
        value: filter,
        onChange: event => setFilter(event.target.value)
      }),
      createElement("ul", null,
        visible.map(user => createElement("li", { key: user.id },
          createElement(Badge, { name: user.name }),
          " & friends"))),
      (visible.length === 0) && createElement("p", null, "No users")));
});

function renderName(name) {
  const upper = name.toUpperCase();
  return createElement("b", null, upper);
}
```

The imports are computed from the model: the hooks, `memo`, `createContext` and `createElement` from `react`, and `Badge` from its module, by the reference of the JSX element to its declaration. The HTML intrinsics are written as React expects them: `class` as `className`, `for` as `htmlFor`, the style as an object with camel-cased properties, and the static value of the input dropped in favor of the JSX attribute that controls it. The test compares both outputs with these files.

6.2. HTML, shared

An `Intrinsic` holds an element of the HTML model, interpreted by the same structure the HTML generator uses. A page model and a component model therefore mean the same markup, and HTML built by a transformation, a Bootstrap layout of the runtime's HTML Bootstrap model for example, can be used in a component as it is: `new ReactGenerator(new BootstrapStructure())` renders Bootstrap elements as intrinsics.

6.3. The generator

`ReactGenerator` is a TypeScript generator that also writes components with hooks, contexts and JSX. The syntax is chosen per module: JSX in a `TsxModule`, written to `.tsx` and compiled with the automatic runtime, and `createElement` calls in any other module, which is plain JavaScript when it has no type annotations. A component declared with `memo` is wrapped in React's `memo`; a component's documentation becomes JSDoc.

Chapter 7. Component libraries

A component of a library, such as a React Bootstrap card or the React Flow canvas, is a class of a model annotated with its component's name and module: a `LibraryComponent`. The generator renders its features as props, reflectively:

- attributes as values;
- containments of JSX as children;
- value objects as object literals;
- enumeration literals as values, or as members of an enumeration the library exports.

Expressions, event handlers and children that are computed are JSX attributes and children of the library component, as for any JSX element. A component library is therefore annotated Xcore, with no generator code, which is how the React Bootstrap and React Flow models are written.

7.1. React Bootstrap

The React Bootstrap model describes the components of React Bootstrap 2.10, for Bootstrap 5.3, in five packages: layout and the shared vocabulary, content, navigation, overlays, and forms. Like the HTML Bootstrap model, it describes what a component means: a container fluid until a breakpoint, a column 4 wide with an offset of 2 from the medium breakpoint.

Listing 12. A grid in React Bootstrap (`BootstrapGeneratorTests.java`, in the React Bootstrap model)

```
var container = with(CORE.createContainer(), c -> c.setFluidUntil(Breakpoint.MD));
var row = with(CORE.createRow(), r -> {
    r.getColumnsPerRow().add(with(CORE.createRowColumns(), rc -> {
        rc.setBreakpoint(Breakpoint.XS);
        rc.setCount(1);
    }));
    r.getColumnsPerRow().add(with(CORE.createRowColumns(), rc -> {
        rc.setBreakpoint(Breakpoint.MD);
        rc.setCount(3);
    }));
    r.getClasses().addAll(List.of("g-3", "mb-3"));
});
container.getRows().add(row);
row.getColumns().add(with(CORE.createCol(), c -> {
    c.getSizes().add(size(Breakpoint.XS, 12));
    c.getSizes().add(size(Breakpoint.MD, 4, 2));
    c.getChildren().add(Bootstrap.text("First"));
}));
```

```
row.getColumns().add(with(CORE.createCol(), c -> {
    c.getSizes().add(with(CORE.createColumnSize(), s -> {
        s.setBreakpoint(Breakpoint.MD);
        s.setAuto(true);
    }));
    c.getStyle().put("min-height", "4rem");
    c.getChildren().add(with(CORE.createStack(), s -> {
        s.setDirection(Direction.HORIZONTAL);
        s.setGap(2);
        s.getChildren().add(Bootstrap.text("Stacked"));
    }));
}));

assertEquals("""
        import { Container, Row, Col, Stack } from "react-bootstrap";

        function Demo() {
          return (
            <Container fluid="md">
              <Row className="g-3 mb-3" xs={1} md={3}>
                <Col xs={12} md={{ span: 4, offset: 2 }}>First</Col>
                <Col style={{ minHeight: "4rem" }} md="auto">
                  <Stack direction="horizontal" gap={2}>Stacked</Stack>
                </Col>
              </Row>
            </Container>
          );
        }
        """, generate(container, true));
```

The components are imported from `react-bootstrap` because their classes say so, and responsive values become React Bootstrap's props: `md={{ span: 4, offset: 2 }}`.

Bootstrap thus appears twice in the runtime, for two ways of delivering a page: the HTML Bootstrap model for pages written as HTML, and the React Bootstrap model for interactive applications.

7.2. React Flow

The React Flow model describes `@xyflow/react` 12: a flow of nodes and edges, the `ReactFlow` component with its background, controls, minimap and panels, the hooks that hold nodes and edges in state, and the building blocks of custom nodes.

Listing 13. An editable flow (`FlowGeneratorTests.java`, in the React Flow model)

```
var a = node("a", "A", 0, 0);
var b = node("b", "B", 200, 0);
```

```
ComponentDeclaration component = with(REACT.createComponentDeclaration(), c -> {
    c.setName("Editor");
    c.getHooks().add(with(FLOW.createNodesState(), h -> {
        h.getBindings().addAll(java.util.List.of("nodes", "setNodes", "onNodesChange"));
        h.getInitialNodes().add(a);
        h.getInitialNodes().add(b);
    }));
    c.getHooks().add(with(FLOW.createEdgesState(), h -> {
        h.getBindings().addAll(java.util.List.of("edges", "setEdges", "onEdgesChange"));
        h.getInitialEdges().add(with(edge(a, b), e -> e.setEdgeType(EdgeType.SMOOTH_STEP)));
    }));
    var onConnect = callback(
            "onConnect",
            "connection => setEdges(current => addEdge(connection, current))",
            "setEdges");
    onConnect.getExpression().getReferences().add(external("addEdge", XYFLOW));
    c.getHooks().add(onConnect);
    c.setRender(attributes(
            with(FLOW.createReactFlow(), f -> {
                f.setColorMode(ColorMode.DARK);
                f.setConnectionLineType(ConnectionLineType.SMOOTH_STEP);
                f.setDefaultEdgeOptions(with(FLOW.createEdgeOptions(), o -> {
                    o.setAnimated(true);
                    o.setMarkerEnd(FLOW.createEdgeMarker());
                }));
                f.getDeleteKeyCode().addAll(java.util.List.of("Backspace", "Delete"));
            }),
            attribute("nodes", "nodes"),
            attribute("edges", "edges"),
            event("onNodesChange", "onNodesChange"),
            event("onEdgesChange", "onEdgesChange"),
            event("onConnect", "onConnect")));
});
Expected.assertMatches("controlled.tsx", generate(module(component)));
```

Listing 14. `controlled.tsx`

```
import { addEdge, useNodesState, useEdgesState, ReactFlow, ConnectionLineType, MarkerType } from
"@xyflow/react";
import { useCallback } from "react";

export function Editor() {
  const [nodes, setNodes, onNodesChange] = useNodesState([
    { id: "a", position: { x: 0, y: 0 }, data: { label: "A" } },
    { id: "b", position: { x: 200, y: 0 }, data: { label: "B" } }
  ]);
  const [edges, setEdges, onEdgesChange] = useEdgesState([{ id: "a-b", source: "a", target: "b",
type: "smoothstep" }]);
  const onConnect = useCallback(connection => setEdges(current => addEdge(connection, current)),
[setEdges]);
  return (
    <ReactFlow
      colorMode="dark"
```

```
      deleteKeyCode={["Backspace", "Delete"]}
      connectionLineType={ConnectionLineType.SmoothStep}
      defaultEdgeOptions={{ animated: true, markerEnd: { type: MarkerType.Arrow } }}
      nodes={nodes}
      edges={edges}
      onNodesChange={onNodesChange}
      onEdgesChange={onEdgesChange}
      onConnect={onConnect}
    />
  );
}
```

The initial nodes and edges are object literals from model objects, the type of the connection line and the marker are members of the library's enumerations, and `addEdge`, used in opaque text, is imported because the expression's references name it.

`LayeredLayout` places the nodes of a flow in layers: a node without outgoing edges in the first, and any other node one layer above the highest node it points to, so for dependencies the dependents are on top. It is readable, not good, on purpose: a layout to start from, which a user rearranges, or a layout library replaces.

Chapter 8. Applications

An application generated from these models is three kinds of files: a page, ES modules, and data.

- The page is a document of the runtime's HTML model, with an import map that resolves bare specifiers such as `react` and `react-bootstrap` to a CDN, esm.sh, and a module script that mounts the root component. The import map is the only place the CDN is named.
- The modules are written by `ReactGenerator`, as plain JavaScript with `createElement` calls, so the browser runs them as they are.
- The data is JSON, fetched by the components.

8.1. Static or served

The files can be written to a directory, which any static host serves, and React makes the application dynamic with no JVM behind it. ES modules do not load from `file:` URLs, so even on a laptop the files need an HTTP server, such as the JDK's `jwebserver`.

The same files can be served from the JVM, by the runtime's HTTP handlers, with the data computed for each request. The page and the modules use relative URLs, so they are the same bytes in both modes: a preview is the deployment.

The tests of the React model generate four applications this way, from Java, with no npm and no build step:

- a documentation viewer, with a navigation tree, Markdown content, search and Bootswatch themes;
- an interactive table of the running JVM's modules, with filtering, sorting and a choice of columns;
- a table of the running JVM's properties;
- a diagram of module dependencies in React Flow, which the user rearranges, highlights and edits.

They are written to `target/react`.

8.2. Walking through an application

A generated application is checked the way a user checks it: in a browser. The runtime's `playwright` module drives one with Playwright for Java, against a server of its `harness` module, and the whole session is recorded as one trace of the runtime's telemetry model:

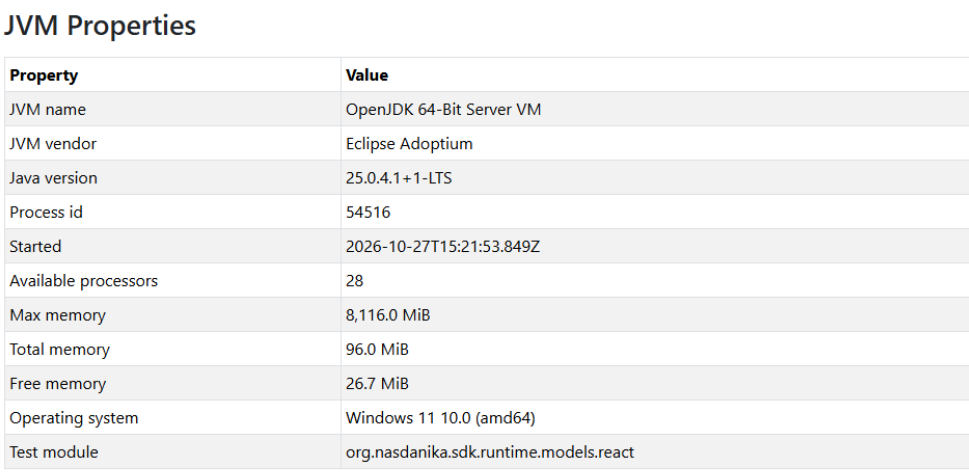
Listing 15. A walk-through of the JVM properties table (`JvmPropertiesWalkthroughTests.java`)

```
Recording recording = Recording.builder()
        .serviceName("jvm-properties")
        .directory(dir, "yml")
        .build();
try (recording) {
    OpenTelemetry otel = recording.getOpenTelemetry();
    Harness.inServer(otel, "JVM properties", JvmPropertiesTableTests::routes, server -> ①
        Browsing.inBrowser(server, options(dir), session -> {
            Page page = session.page();
            session.step("Open the JVM properties", () -> {
                page.navigate("/index.html");
                page.getByText("Process id").waitFor(); ②
            });
            session.step("Reload for fresh numbers", () -> {
                page.reload();
                page.getByText("Free memory").waitFor();
            });
            return null;
        }));
}
```

① A server with the application's routes: the page, the module and the data. Each request is a span.

② The step waits for the table to render the data.

Each step runs in a span, with screenshots before and after it recorded as log records with the images' paths. Every request the browser sends to the server carries the step's span, so the server's spans for the page, the module and the data are children of the step that caused them. The test checks exactly that, and the screenshot after the first step is this book's figure:



JVM Properties

Property	Value
JVM name	OpenJDK 64-Bit Server VM
JVM vendor	Eclipse Adoptium
Java version	25.0.4.1+1-LTS
Process id	54516
Started	2026-10-27T15:21:53.849Z
Available processors	28
Max memory	8,116.0 MiB
Total memory	96.0 MiB
Free memory	26.7 MiB
Operating system	Windows 11 10.0 (amd64)
Test module	org.nasdanika.sdk.runtime.models.react

Figure 1. The JVM properties table, generated from Java models, served, and opened in Chrome by a test

A recorded walk-through is a user guide and a demo video in waiting: steps, pictures, and what the server did, produced again with every build. The walk-through needs a browser and access to esm.sh, and is skipped without them.

Part III: Looking Ahead

Interfaces drawn once and rendered many ways, a user interface for any model, and vocabularies as npm packages.

Chapter 9. Looking ahead

Everything in this chapter is **PLANNED**.

9.1. A UI model, drawn

A model of user interfaces that does not depend on a framework, drawn in Draw.io with a library of Bootstrap shapes: one drawing, several renderers. Its concepts map onto the React model in a transformation, not by inheritance, because React is a technology of rendering, and a drawn interface should not take React's shape while it is being drawn.

9.2. The polymorphic UI

A browser application that renders any model, and live telemetry, from the runtime's HTTP server: views chosen by the class of the element they show and the format asked for, generated with these models.

9.3. Vocabularies as npm packages

Transformations into the TypeScript model from EMF packages and from vocabularies of the runtime's Meta model, with type information at run time, published as npm packages with their data (Chapter 1).



> The module `models/typescript/ecore`, which generates TypeScript packages from EMF packages, is commented out of the build and still uses the previous generation's package `org.nasdanika.models.typescript.ecore` and module `org.nasdanika.common`.

9.4. One source, many audiences

Generated output often has to vary by coordinate: jurisdiction, language, tier, audience. Which elements apply where belongs to the source model, and the request to generate for a coordinate to the transformation; only the receipt belongs to the TypeScript model: a `Module` will record the coordinate that produced it, next to its markers. Generic packages are the published default, and variants are generated on demand for consumers who exist.

9.5. Vendored modules

React and the libraries copied into the application’s files, with the import map pointing at the copies, for offline use and closed networks.

9.6. Documentation for npm users

Generated documentation of packages for people who use npm: what the modules are for, shown in TypeScript, with EMF mentioned rarely.

Appendix A: Reference

A.1. Packages

Package	Namespace URI
`org.nasdanika.sdk.runtime.models.typescript`	`runtime.sdk.nasdanika.org/models/typescript`
`org.nasdanika.sdk.runtime.models.react`	`runtime.sdk.nasdanika.org/models/react`
`org.nasdanika.sdk.runtime.models.react.bootstrap` and its `forms` and `components` packages	`runtime.sdk.nasdanika.org/models/react/bootstrap` and under it
`org.nasdanika.sdk.runtime.models.react.flow`	`runtime.sdk.nasdanika.org/models/react/flow`

A.2. Enumerations of the TypeScript model

Enumeration	What it says
`ImportKind`	Named, default or namespace import
`PrimitiveType`	`string`, `number`, `boolean`, `bigint`, `void` and the other primitive types
`LiteralKind`	The kind of a literal type
`VariableKind`	`const`, `let` or `var`
`Visibility`	`public`, `protected` or `private`
`AccessorKind`	`get` or `set`

A.3. Java classes

Class	What it is for
`typescript.generator.Generator`	Writes a package, with `package.json` and `tsconfig.json`, or a module, with managed imports, to a volume of text or to a string

Class	What it is for
`react.generator.ReactGenerator`	A TypeScript generator that also writes components, hooks, contexts and JSX, as JSX in a `TsxModule` and as `createElement` calls elsewhere. `new ReactGenerator(structure)` renders HTML intrinsics with an HTML structure, such as `BootstrapStructure`
`react.flow.layout.LayeredLayout`	A simple layered layout of a flow

Appendix B: Building this book

This book is AsciiDoc in `models/react/model/src/docs/asciidoc`, built with the React model after its tests have passed:

```
mvn -pl models/react/model -am verify                          # the models, their tests and this
book
mvn -pl models/react/model verify -Dzephyr.editor-notes=false  # without editor's notes
mvn -pl models/react/model verify -Dasciidoctor.skip=true      # without the book
```

The PDF and the single-page HTML are written to `target/book` and attached to the React model's artifact with the classifier `book`; the test sources are attached with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests of four modules: the React model's in `src/test/java/org/nasdanika/sdk/runtime/models/react/tests`, the TypeScript model's book test in `models/typescript/model`, and the generator tests of `models/react/bootstrap` and `models/react/flow`, and of the module declarations. A listing line holds at most 96 characters, and a warning fails the build. Build the TypeScript, React Bootstrap and React Flow models before the book, so that their tests have checked what it includes: `-am` builds the TypeScript model; add `-pl models/react/bootstrap,models/react/flow` for the others.

B.1. Generated code

The generated code after each listing is a file that a test compares with what the generator writes: `src/test/resources/expected` in the React and React Flow models, and `src/test/resources/org/nasdanika/sdk/runtime/models/typescript/tests/book` in the TypeScript model. The tests also write what they generated to `target/expected-actual`, or `target/book-actual` in the TypeScript model; to change an expectation, review the difference, and copy the actual file over the expected one.

B.2. Compiling the generated TypeScript

`BankTests` in the TypeScript model generates a package that uses every construct of the model into `models/typescript/model/target/bank`. The Maven build does not depend on Node.js; compiling the package is a separate step:

```
cd models/typescript/model/target/bank
npm install
npx tsc --noEmit --verbatimModuleSyntax --isolatedModules
npm run build
```

The test wipes only the generated `src` directory, so an `npm install` survives the next test run, but not `mvn clean` of the TypeScript model.



> The React model's POM configures the clean plugin to leave `target/bank` alone, with a comment about `BankTests` and `node_modules`; `BankTests` is in the TypeScript model, whose POM has no such configuration. Move the configuration to the TypeScript model's POM, or remove it.

B.3. The figure

The figure, `jvm-properties.png` in `src/test/resources/org/nasdanika/sdk/runtime/models/react/tests/book`, is the screenshot that `JvmPropertiesWalkthroughTests` writes to `target/react/jvm-properties-walkthrough/001-open-the-jvm-properties-after.png`, copied. To change it, run the test, check that the screenshot shows nothing private, and copy it again.

Appendix C: Glossary

Bare specifier

A module specifier that is a package name, such as `react`, resolved by an import map in a browser or by `node_modules` in Node.js.

ES module

A JavaScript module with `import` and `export`, which browsers load natively.

esm.sh

A CDN that serves npm packages as ES modules.

Import map

A JSON map in a page from bare specifiers to URLs.

Intrinsic

A JSX element that is an HTML element, such as `<div>`, as opposed to a component.

JSX

The syntax of React for elements in JavaScript and TypeScript, compiled to `createElement` calls or to the automatic runtime.

Library component

A component of a library, such as React Bootstrap's, modeled as an annotated class whose features are rendered as props.

NodeNext

TypeScript's module resolution for Node.js's ES modules, under which a relative specifier carries the `.js` extension.

Type-only import

`import type`, erased when compiled: for names used only in type positions.

Walk-through

A recorded session of a browser driven through an application, step by step, with screenshots and the server's spans.